



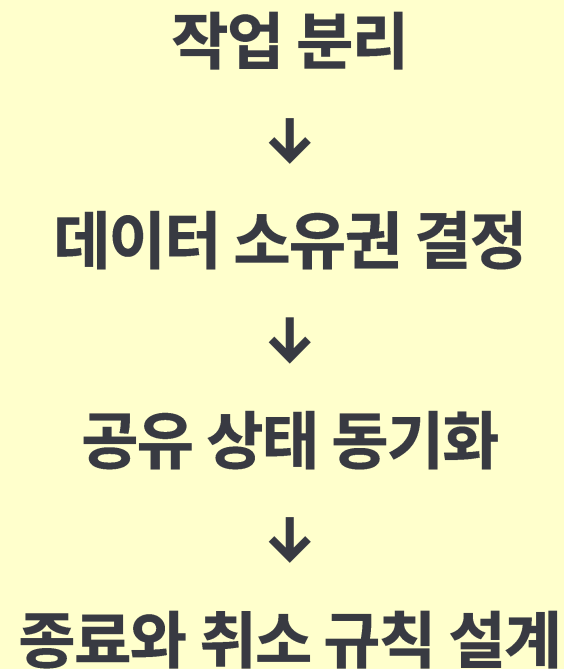
16. 동시성 프로그래밍

Heesung Oh

<https://github.com/3dapi>



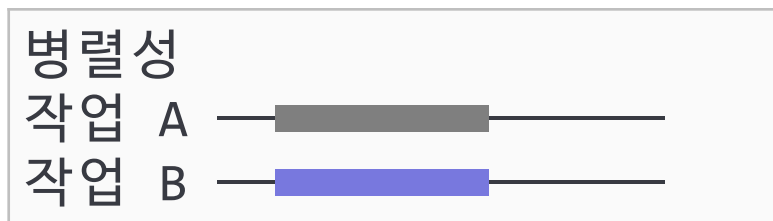
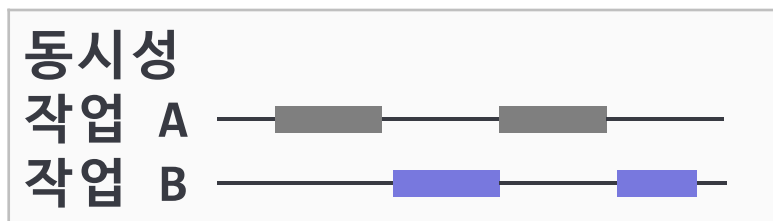
- 동시성, 병렬성
- 프로세스와 스레드
- 공유 상태와 데이터 경쟁
- `std::thread`, `std::jthread`
- 뮤텝스와 RAII 잠금
- 교착 상태와 잠금 순서
- 조건 변수와 작업 큐
- `std::future`, `std::promise`, `std::async`
- `std::atomic`
- C++20 동기화 도구
- 병렬 알고리즘 개요
- 동시성 설계 기준





16.1 동시성과 병렬성

- 동시성: 여러 작업의 실행 구간이 겹치도록 구성
- 병렬성: 여러 작업이 실제로 같은 시간에 실행
- 한 처리 장치에서도 작업 전환으로 동시성 가능
- 실제 병렬 실행에는 둘 이상의 실행 자원 필요
- 스레드 수와 실제 병렬 실행 수는 같다고 보장되지 않음



구분	의미
동시성	작업을 겹쳐 진행하는 구조
병렬성	여러 작업이 동시에 실행되는 상태



구분	프로세스	스레드
주소 공간	일반적으로 분리	같은 프로세스 안에서 공유
데이터 교환	프로세스 간 통신 필요	같은 메모리에 직접 접근 가능
격리	비교적 강함	한 스레드 오류가 전체에 영향 가능
생성·전환 비용	상대적으로 큼	상대적으로 작음

프로세스

- 코드
- 전역 데이터
- 힙
- 스레드 A – 호출 스택
- 스레드 B – 호출 스택

- 같은 프로세스의 스레드는 메모리를 쉽게 공유
- 공유가 쉬운 만큼 접근 규칙 필요
- 동기화 없이 공유 가변 상태를 다루면 위험





```
int counter = 0;

void Increase()
{
    for (int i = 0; i < 100'000; ++i)
    {
        ++counter;
    }
}
```

- ++counter는 하나의 기계적 원자 연산이라고 보장되지 않음
- 읽기, 증가, 기록 과정으로 나뉠 수 있음
- 두 스레드가 같은 값을 읽으면 증가가 사라질 수 있음

counter = 10
스레드 A: 10 읽기
스레드 B: 10 읽기
스레드 A: 11 기록
스레드 B: 11 기록

- 같은 메모리 위치 접근
- 하나 이상이 쓰기
- 적절한 동기화 없음
- 데이터 경쟁 발생
- C++ 프로그램 동작 보장 불가



16.1.4 메모리 가시성과 happens-before

```
int result = 0;  
bool ready = false;
```

```
void Produce()  
{  
    result = 100;  
    ready = true;  
}
```

```
void Consume()  
{  
    if (ready)  
    {  
        Use(result);  
    }  
}
```

- 두 변수에 대한 동기화 없음
- 기록 순서와 관찰 시점 보장 없음
- 컴파일러와 처리 장치의 재배치 가능

```
std::mutex mutex;  
int result = 0;  
bool ready = false;
```

```
void Produce()  
{  
    std::lock_guard lock{mutex};  
    result = 100;  
    ready = true;  
}
```

```
void Consume()  
{  
    std::lock_guard lock{mutex};  
  
    if (ready)  
    {  
        Use(result);  
    }  
}
```

- 같은 뮤텍스의 잠금과 해제
- 접근 직렬화
- 메모리 가시성 연결



- <thread> 헤더
- 하나의 실행 스레드를 나타내는 이동 전용 객체
- 생성자에 호출 대상과 인수 전달
- 생성이 완료되면 새 실행 흐름 시작 가능
- 실행 순서는 정해져 있지 않음

```
#include <iostream>
#include <thread>

void PrintMessage()
{
    std::cout << "worker thread\n";
}

int main()
{
    std::thread worker(PrintMessage);
    worker.join();
}
```

```
int main()
{
    std::thread worker {
        [] {
            std::cout << "lambda thread\n";
        }
    };
    worker.join();
}
```



16.2.2 인수 전달과 객체 수명

- `std::thread`는 함수와 인수를 내부에 저장
- 일반 인수는 복사 또는 이동 저장
- 참조 전달은 `std::ref()`, `std::cref()` 사용
- 참조 대상은 스레드가 끝날 때까지 살아 있어야 함

```
void SetValue(int& target, int value) {  
    target = value;  
}
```

```
int number = 0;  
std::thread worker { SetValue, std::ref(number), 100};  
worker.join();
```

```
void PrintText(const std::string& text);
```

```
std::string message = "loading";
```

```
std::thread worker{PrintText, std::cref(message)};  
worker.join();
```

- 지역 객체 참조 캡처 후 스레드 반환 금지
- 값 캡처 또는 소유 객체 이동으로 수명 분리



16.2.3 join, joinable, detach

- `join()`은 대상 스레드 종료까지 현재 스레드 대기
- `join()` 후 `std::thread`는 실행 스레드를 나타내지 않음
- `joinable()`로 결합 가능한 상태 확인
- `detach()`는 스레드 객체와 실행 스레드 연결 해제
- 분리된 스레드는 다시 `join()` 불가

```
std::thread worker{RunTask};

if (worker.joinable())
{
    worker.join();
}

std::thread first{RunTask};
std::thread second{std::move(first)};

if (!first.joinable())
{
    std::cout << "first is empty\n";
}

second.join();
```

- 종료 시점과 참조 대상 수명을 보장하기 어렵다면 `detach()` 지양



- 결합 가능한 `std::thread` 객체가 소멸하면 `std::terminate()` 호출
- 예외가 발생해 `join()`을 건너뛰어도 같은 문제
- 스레드 함수 밖으로 예외가 빠져나와도 `std::terminate()` 호출

```
void Process()  
{  
    std::thread worker{RunTask};  
  
    PerformOperation();  
  
    worker.join();  
}
```

PerformOperation() 예외



worker.join() 건너뛰



결합 가능한 `std::thread` 소멸



`std::terminate()`

- 모든 정상 경로와 예외 경로에서 스레드 수명 정리 필요
- C++20 `std::jthread`는 자동 결합 제공



16.2.5 std::jthread (C++20)

- std::thread와 비슷하게 스레드 시작
- 소멸 시 결합 가능한 스레드에 중지 요청
- 이후 join() 수행
- 범위 기반 스레드 수명 관리에 적합

```
#include <iostream>
#include <thread>

void PrintMessage()
{
    std::cout << "jthread worker\n";
}

int main()
{
    std::jthread worker{PrintMessage};
}
```

main 종료
↓
worker 소멸
↓
중지 요청
↓
join()

- 자동 결합은 작업을 강제로 중단하지 않음
- 작업 함수가 반환해야 소멸 완료



16.2.6 stop_token을 이용한 협력적 중지

```
#include <chrono>
#include <iostream>
#include <stop_token>
#include <thread>

using namespace std::chrono_literals;

void Worker(std::stop_token stopToken)
{
    while (!stopToken.stop_requested())
    {
        std::cout << "working\n";
        std::this_thread::sleep_for(100ms);
    }

    std::cout << "stopped\n";
}
```

```
int main()
{
    std::jthread worker{Worker};

    std::this_thread::sleep_for(250ms);
    worker.request_stop();
}
```

- request_stop()은 중지 상태 설정
- 실행 중인 코드를 강제 중단하지 않음
- 작업 함수가 stop_requested() 확인
- 처리 중 작업을 완료할지 폐기할지 정책 필요



- 뮤텍스는 상호 배제 제공
- 같은 임계 구역을 여러 스레드가 동시에 실행하지 못하게 함
- 공유 데이터에 접근하는 모든 코드가 같은 뮤텍스 사용 필요
- 수동 lock()/unlock()은 조기 반환과 예외에 취약

```
#include <mutex>
```

```
int counter = 0;
```

```
std::mutex counterMutex;
```

```
void Increase()
```

```
{
```

```
    for (int i = 0; i < 100'000; ++i)
```

```
    {
```

```
        counterMutex.lock();
```

```
        ++counter;
```

```
        counterMutex.unlock();
```

```
    }
```

```
}
```

```
counterMutex.lock();
```

```
if (!CanUpdate())
```

```
{
```

```
    return; // unlock() 누락
```

```
}
```

```
counterMutex.unlock();
```



16.3.2 std::lock_guard

- 생성 시 뮤텝스 잠금
- 소멸 시 뮤텝스 해제
- 범위 종료, 조기 반환, 예외 경로에서 해제 보장
- 단순한 한 범위 잠금에 적합

```
void Increase()
{
    for (int i = 0; i < 100'000; ++i)
    {
        std::lock_guard lock{counterMutex};
        ++counter;
    }
}
```

```
class Account
{
public:
    void Deposit(int amount)
    {
        std::lock_guard lock{mutex};
        balance += amount;
        ++transactionCount;
    }
private:
    mutable std::mutex mutex;
    int balance = 0;
    int transactionCount = 0;
};
```

- 여러 값을 하나의 불변식으로 관리하면 같은 잠금 범위에서 변경



16.3.3 std::scoped_lock

- 하나 이상의 뮤텝스를 잠금
- 여러 뮤텝스를 교착 상태를 피하는 방식으로 잠금
- 잠금 객체 소멸 시 모두 해제
- 계좌 이체처럼 두 객체를 함께 보호할 때 유용

```
class Account
{
public:
    friend void Transfer(Account& from, Account& to, int amount);

private:
    std::mutex mutex;
    int balance = 0;
};
```

```
void Transfer(Account& from, Account& to, int amount) {
    if (&from == &to) {
        return;
    }
    std::scoped_lock lock{from.mutex, to.mutex};

    if (from.balance < amount) {
        return;
    }
    from.balance -= amount;
    to.balance += amount;
}
```



- 잠금 소유 여부를 상태로 보관
- 이동 가능
- 지연 잠금 가능
- 명시적 해제와 재 잠금 가능
- 조건 변수 대기 필요

잠금 도구	적합한 상황
lock_guard	단순한 한 범위 잠금
scoped_lock	여러 뮤텝스 함께 잠금
unique_lock	잠금 상태 제어, 조건 변수

```
std::unique_lock lock{counterMutex};  
  
++counter;  
  
lock.unlock();  
  
PerformLongOperation();  
  
lock.lock();  
  
ReadCounter(counter);  
  
std::unique_lock lock{  
    counterMutex,  
    std::defer_lock};  
  
PrepareLocalData();  
  
lock.lock();  
  
UpdateSharedData();
```





```
void Update()
{
    std::lock_guard lock{mutex};

    if (!IsValid())
    {
        return;
    }

    ModifyState();
}
```

- 함수가 정상 종료되어도 잠금 해제
- 조기 반환되어도 잠금 해제
- 예외로 빠져나가도 잠금 해제
- 잠금 해제 보장은 RAII가 담당

Data snapshot;

```
{
    std::lock_guard lock{mutex};
    snapshot = sharedData;
}

SaveToFile(snapshot);
```

- 공유 데이터 복사는 잠금 안에서 수행
- 파일 입출력 같은 긴 작업은 잠금 밖에서 수행
- 잠금 범위는 짧게
- 상태 변경 원자성은 깨지지 않게





- 둘 이상의 실행 흐름이 서로가 가진 자원을 기다림
- 어느 쪽도 진행하지 못함
- 프로그램이 종료되지 않고 멈춘 것처럼 보일 수 있음

조건	의미
상호 배제	한 자원을 한 스레드만 사용
점유와 대기	자원을 가진 채 다른 자원 대기
비 선점	다른 스레드 자원을 강제로 빼앗을 수 없음
순환 대기	원형으로 서로의 자원 대기

```
void FirstTask()  
{  
    std::lock_guard firstLock{firstMutex};  
    std::lock_guard secondLock{secondMutex};  
}  
void SecondTask()  
{  
    std::lock_guard secondLock{secondMutex};  
    std::lock_guard firstLock{firstMutex};  
}
```

- 반대 순서 잠금은 순환 대기 가능



- 모든 코드에서 뮤텁스를 같은 순서로 잠금
- 순환 대기 제거
- 대규모 코드에서는 잠금 계층과 순서 문서화
- 직접 순서 관리가 어렵다면 표준 도구 사용

// 여러 뮤텁스 안전 잠금

```
void Exchange(Data& left, Data& right)
```

```
{
```

```
    if (&left == &right)
```

```
    {
```

```
        return;
```

```
    }
```

```
    std::scoped_lock lock{left.mutex, right.mutex};
```

```
    std::swap(left.value, right.value);
```

```
}
```





- 잠금을 오래 보유하면 경합 증가
- 공유 상태 접근에 필요한 범위만 보호
- 잠금 안에서 외부 콜백이나 가상 함수 호출 지양
- 호출 대상이 어떤 잠금을 획득할지 알기 어려움

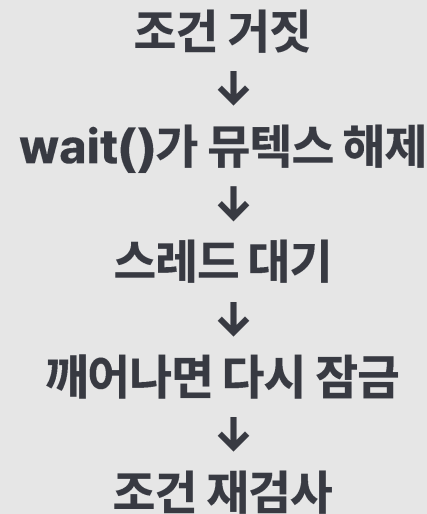
```
Task task;  
  
{  
    std::lock_guard lock{mutex};  
    task = tasks.front();  
    tasks.pop();  
}  
  
Execute(task);
```

```
std::function<void()> localCallback;  
  
{  
    std::lock_guard lock{mutex};  
    localCallback = callback;  
}  
  
if (localCallback)  
{  
    localCallback();  
}
```



- 공유 상태가 특정 조건을 만족할 때까지 스레드 대기
- 바쁜 대기 방지
- `<condition_variable>` 헤더
- 대기에는 `std::unique_lock<std::mutex>` 필요
- 조건 변수의 핵심은 통지가 아니라 공유 상태

```
std::mutex mutex;  
std::condition_variable condition;  
std::queue<Task> tasks;  
bool stopped = false;  
  
std::unique_lock lock{mutex};  
  
condition.wait(lock, [&  
{  
    return !tasks.empty() || stopped;  
});
```





- `wait()`는 통지 없이 반환할 수 있음
- 가짜 깨어남 가능
- 통지를 받았어도 다른 소비자가 먼저 작업을 가져갈 수 있음
- 조건식은 반복해서 검사해야 함

```
condition.wait(lock, [&]  
{  
    return !tasks.empty() || stopped;  
});  
  
while (tasks.empty() && !stopped)  
{  
    condition.wait(lock);  
}
```

함수	의미
<code>notify_one()</code>	대기 스레드 하나 깨움
<code>notify_all()</code>	대기 스레드 모두 깨움

- 작업 하나 추가 → 보통 `notify_one()`
- 종료 상태 변경 → 보통 `notify_all()`
- 공유 상태를 먼저 변경하고 통지



- 작업을 만드는 스레드와 처리하는 스레드를 분리
 - ◆ 생산자 : [작업 생성] —▶ [공유 작업 큐에 저장]
 - ◆ 소비자 : [공유 작업 큐 확인] —▶ [작업 꺼내기] —▶ [작업 처리]

```
// 생산자, 소비자 태스크.  
void TaskQueue::Push(int value)  
{  
    {  
        std::lock_guard lock{mutex};  
        tasks.push(value);  
    }  
  
    // 대기 중인 스레드에 큐 상태 변경을 통지.  
    condition.notify_one();  
}
```

```
bool WaitPop(int& value)  
{  
    std::unique_lock lock{mutex};  
  
    // 작업이 들어오거나 종료 상태가 될 때까지 대기.  
    condition.wait(lock, [&] {  
        return !tasks.empty() || stopped;  
    });  
  
    if (tasks.empty())  
    {  
        return false;  
    }  
    value = tasks.front();  
    tasks.pop();  
    return true;  
}
```



- wait_for()는 상대 시간까지 대기
- wait_until()은 절대 시점까지 대기
- 조건식 오버로드의 반환값은 조건 만족 여부
- 시간 초과와 상태 변경이 거의 동시에 발생할 수 있음
- 최종 판단은 잠금 안에서 공유 상태 확인

```
using namespace std::chrono_literals;

bool ready = condition.wait_for(lock, 500ms, [&] {
    return !tasks.empty() || stopped;
});

if (!ready)
{
    std::cout << "timeout\n";
}
```

- 조건식 없는 상대 시간 반복 대기는 가짜 깨어남마다 제한 시간이 다시 시작
- 절대 종료 시점 또는 조건식 오버로드 사용



- 스레드의 계산 결과를 호출한 쪽으로 전달
- 결과와 예외를 공유 상태로 전달
- `std::future<T>`는 나중에 준비될 결과 참조
 - ◆ 여러 곳에서 같은 결과를 읽을 때 사용
- `get()`은 결과 준비까지 대기
- 작업 예외는 `get()`에서 다시 발생
- `get()`은 보통 한 번만 호출 가능

```
#include <future>
#include <iostream>

int Calculate()
{
    return 40 + 2;
}

int main()
{
    std::future<int> result =
        std::async(
            std::launch::async,
            Calculate);

    std::cout << result.get() << '\n';
}
```



- 생산자가 공유 상태에 값 또는 예외 설정
- 소비자는 get_future()로 연결된 future 획득
- std::promise는 이동 전용
- 값을 설정하지 않고 소멸하면 broken promise 오류 가능

```
#include <future>
#include <iostream>
#include <thread>

void Produce(std::promise<int> promise)
{
    promise.set_value(100);
}

int main()
{
    std::promise<int> promise;
    std::future<int> result = promise.get_future();

    std::jthread worker{Produce, std::move(promise)};

    std::cout << result.get() << '\n';
}
```

```
void Produce(std::promise<int> promise) {
    try {
        promise.set_value(Calculate());
    }
    catch (...) {
        promise.set_exception(std::current_exception());
    }
}
```



- 호출 가능한 객체와 future 공유 상태 연결
- task를 호출하면 반환값 또는 예외가 공유 상태에 저장
- 스레드를 직접 만들지 않음
- 어느 스레드에서 언제 호출할지는 프로그램이 결정



16.6.4 std::async와 실행 정책

정책	의미
std::launch::async	별도 실행 스레드에서 작업 요구
std::launch::deferred	get() 또는 wait() 호출 스레드에서 실행
정책 생략	구현이 실행 방식 선택

```
auto result = std::async(  
    std::launch::async,  
    Calculate);
```

```
auto deferred = std::async(  
    std::launch::deferred,  
    Calculate);
```

- 동시 실행이 의미에 필요하면 std::launch::async 명시
- 반환된 임시 future는 문장 끝에서 대기할 수 있음

```
auto result = std::async(Calculate);
```

```
auto first = std::async(std::launch::async, FirstTask);  
auto second = std::async(std::launch::async, SecondTask);
```

```
first.get();  
second.get();
```



```
auto result = std::async(
    std::launch::async,
    []() -> int
    {
        throw std::runtime_error{"calculation failed"};
    });

try
{
    int value = result.get();
}
catch (const std::exception& exception)
{
    std::cout << exception.what() << '\n';
}
```

```
using namespace std::chrono_literals;

if (result.wait_for(100ms) == std::future_status::ready)
{
    std::cout << result.get() << '\n';
}
```

함수	의미
get()	결과 또는 예외 획득
wait()	준비될 때까지 대기
wait_for()	상대 시간 동안 준비 상태 확인
wait_until()	절대 시점까지 준비 상태 확인





- 한 객체에 대한 읽기와 쓰기를 원자적으로 수행
- 다른 스레드가 연산 중간 상태를 관찰하지 못함
- 단일 값의 독립적인 상태 변화에 적합
- <atomic> 헤더
- std::atomic은 복사 불가

```
#include <atomic>
#include <iostream>
#include <thread>

std::atomic<int> counter = 0;

void Increase()
{
    for (int i = 0; i < 100'000; ++i)
    {
        ++counter;
    }
}
```

```
int main()
{
    std::jthread first{Increase};
    std::jthread second{Increase};

    first.join();
    second.join();

    std::cout << counter.load() << '\n';
}
```



16.7.2 원자적 읽기, 쓰기와 RMW

```
std::atomic<bool> ready = false;
```

```
ready.store(true);
```

```
bool state = ready.load();
```

```
bool wasReady = ready.exchange(true);
```

```
std::atomic<int> count = 0;
```

```
int oldValue = count.fetch_add(1);
```

```
int newValue = ++count;
```

- fetch_add(1)은 증가 전 값 반환
- 전위 ++은 증가 후 값 반환
- 읽기-수정-쓰기 연산 사용

```
std::atomic<int> count = 0;
```

```
count = count + 1;
```

- 읽기와 쓰기는 각각 원자적
- 전체 식이 하나의 원자적 증가가 되지는 않을 수 있음
- 여러 값의 관계에는 뮤텍스가 더 적합



- 현재 값이 기대값과 같을 때 새 값으로 변경
 - ◆ 성공 → true
 - ◆ 실패 → expected에 실제 현재값 기록

```
std::atomic<int> state = 0;
int expected = 0;
bool changed = state.compare_exchange_strong(expected, 1);

void UpdateMaximum(std::atomic<int>& maximum, int value)
{
    int current = maximum.load();
    while (current < value && !maximum.compare_exchange_weak(current, value))
    {
    }
}
```

- compare_exchange_weak()는 값이 같아도 실패 가능
- 반복문에서 사용하기 적합
- 잠금 없는 자료구조는 CAS만으로 끝나지 않음
- 메모리 회수와 ABA 문제까지 고려 필요



- 원자값이 특정 값과 같은 동안 대기
- 값 변경 후 notify_one() 또는 notify_all()로 깨움
- 값 하나의 변화 대기

```
#include <atomic>
#include <iostream>
#include <thread>

std::atomic<int> state = 0;

void Consumer()
{
    state.wait(0);
    std::cout << "state: " << state.load() << '\n';
}
```

```
int main()
{
    std::jthread worker{Consumer};

    state.store(1);
    state.notify_one();
}
```

- 여러 조건과 복합 상태에는 조건 변수와 뮤텝스가 더 분명
- 큐 상태, 종료 플래그, 오류 상태를 함께 검사할 때 조건 변수 우선 검토



메모리 순서	중심 의미
<code>memory_order_seq_cst</code>	가장 강한 전역 순서 제공, 기본값
<code>memory_order_release</code>	앞선 쓰기를 다른 스레드에 게시
<code>memory_order_acquire</code>	<code>release</code> 로 게시된 쓰기 관찰
<code>memory_order_acq_rel</code>	읽기-수정-쓰기에서 <code>acquire</code> 와 <code>release</code> 함께 적용
<code>memory_order_relaxed</code>	원자성은 유지, 다른 메모리 접근 순서 연결 없음

```
int data = 0;
std::atomic<bool> ready = false;
void Produce()
{
    data = 100;
    ready.store(true, std::memory_order_release);
}
void Consume()
{
    while (!ready.load(std::memory_order_acquire))
    {
    }
    std::cout << data << '\n';
}
```

- `acquire`가 `release` 쓰기를 관찰하면 앞선 `data = 100`도 관찰 가능
- 약한 메모리 순서는 정확성 증명 후 제한적으로 사용



16.7.6 뮤텍스와 원자적 연산의 선택

요구	적합한 도구
단일 카운터 증가	<code>std::atomic</code>
중지 플래그	<code>std::atomic<bool></code> 또는 <code>stop_token</code>
간단한 상태 전환	<code>std::atomic</code>
컨테이너 삽입·제거	뮤텍스
여러 값의 일관된 변경	뮤텍스
검사 후 변경을 한 단위로 처리	뮤텍스
긴 상태 전이와 예외 처리	뮤텍스

```
class BoundedValue {  
public:  
    bool Add(int amount)  
    {  
        std::lock_guard lock{mutex};  
        if (value + amount > maximum)  
        {  
            return false;  
        }  
        value += amount;  
        return true;  
    }  
};
```





- 뮤텝스와 목적이 다름
- 공유 데이터 임계 구역 보호보다 작업 진행 시점 조정에 초점

도구	용도	재사용
<code>std::latch</code>	정해진 작업 수의 1회 완료 대기	불가
<code>std::barrier</code>	여러 단계의 동기화 지점	가능
<code>std::counting_semaphore</code>	동시 사용 가능한 자원 수 제한	가능

latch

준비 A }
준비 B } — 모두 준비 → 시작
준비 C }

barrier

단계 1 모두 도착 → 단계 2
단계 2 모두 도착 → 단계 3

semaphore

허가 수만큼만 통과



- 내부 카운터가 0이 될 때까지 대기
- 카운터가 0이 된 뒤 다시 초기화 불가
- 초기화 완료, 여러 작업 시작선에 적합

```
#include <iostream>
#include <latch>
#include <thread>

int main()
{
    std::latch ready{3};
    auto worker = [&](int id)
    {
        std::cout << "prepare " << id << '\n';
        ready.count_down();
        ready.wait();
        std::cout << "start " << id << '\n';
    };
    std::jthread first{worker, 1};
    std::jthread second{worker, 2};
    std::jthread third{worker, 3};
}
```

// count_down()과 wait()를 한 번에 수행
ready.arrive_and_wait();





16.8.2 std::barrier

- 정해진 수의 참여자가 도착할 때까지 대기
- 모두 도착하면 다음 단계로 진행
- 단계마다 재사용 가능
- 단계 완료 함수 지정 가능

```
#include <barrier>
#include <iostream>
#include <thread>

int main()
{
    std::barrier syncPoint{2};
    auto worker = [&](int id)
    {
        for (int phase = 0; phase < 3; ++phase)
        {
            std::cout << id << ": phase " << phase << "\n";
            syncPoint.arrive_and_wait();
        }
    };
    std::jthread first{worker, 1};
    std::jthread second{worker, 2};
}
```

```
std::barrier syncPoint{
    2,
    []() noexcept
    {
        std::cout << "phase completed\n";
    }};
```





16.8.3 std::counting_semaphore

- 동시에 통과할 수 있는 실행 흐름 수 제한
- acquire()는 허가를 얻을 때까지 대기
- release()는 허가 수 증가
- 소유 스레드 개념 없음
- std::binary_semaphore 최대 카운트 1

```
#include <chrono>
#include <iostream>
#include <semaphore>
#include <thread>
using namespace std::chrono_literals;

std::counting_semaphore<2> slots{2};
void UseResource(int id) {
    slots.acquire();

    std::cout << "enter " << id << '\n';
    std::this_thread::sleep_for(100ms);
    std::cout << "leave " << id << '\n';

    slots.release();
}
```

- 최대 두 스레드가 자원 구역에 진입
- 뮤텁스는 잠근 스레드가 해제해야 함



- 일부 표준 알고리즘은 실행 정책 오버로드 제공
- `<execution>` 헤더
- 실행 정책은 허용 범위를 지정
- 실제 병렬 실행 여부와 스레드 수는 구현에 의존

정책	의미
<code>std::execution::seq</code>	순차 실행
<code>std::execution::par</code>	병렬 실행 허용
<code>std::execution::par_unseq</code>	병렬 실행과 벡터화 허용
<code>std::execution::unseq</code>	C++20, 비순차 벡터화 허용

```
#include <algorithm>
#include <execution>
#include <vector>

std::vector<int> values{5, 4, 3, 2, 1};

std::sort(
    std::execution::par,
    values.begin(),
    values.end());
```





- 각 원소 작업이 서로 독립적일 때 적합
- 작업량이 충분히 커야 이점 가능
- 분할, 동기화, 캐시 이동 비용 존재
- 메모리 대역폭 병목이면 효과 제한
- 구현과 빌드 환경의 병렬 알고리즘 지원 확인 필요
- 함수 객체가 공유 상태를 변경하지 않는 구조가 안전

```
std::transform(  
    std::execution::par,  
    source.begin(),  
    source.end(),  
    destination.begin(),  
    [](float value)  
    {  
        return value * value;  
    });
```

원소 0 → 독립 계산 → 결과 0
원소 1 → 독립 계산 → 결과 1
원소 2 → 독립 계산 → 결과 2



16.9.3 부작용, 데이터 경쟁과 예외

```
int total = 0;
```

```
std::for_each(  
    std::execution::par,  
    values.begin(),  
    values.end(),  
    [&](int value)  
    {  
        total += value; // 안전하지 않음  
    });
```

- 공유 변수 동기화 없는 쓰기
- 데이터 경쟁 가능
- 병렬 알고리즘에 맞지 않는 부작용

```
#include <numeric>
```

```
int total = std::reduce(  
    std::execution::par,  
    values.begin(),  
    values.end(),  
    0);
```

- 합계에는 병렬 축약 알고리즘 사용
- reduce()는 연산 순서를 바꿀 수 있음
- 부동소수점 결과 재현성 주의
- 표준 실행 정책에서 사용자 함수 예외가 빠져나오면 std::terminate() 가능



- 병렬화 전 순차 구현 측정
- 같은 입력과 빌드 설정으로 반복 측정
- 한 번의 측정만으로 결론 금지
- 중앙값이나 분포 비교

```
const auto start = std::chrono::steady_clock::now();  
  
RunAlgorithm();  
  
const auto end =  
    std::chrono::steady_clock::now();  
const auto elapsed =  
    std::chrono::duration_cast<std::chrono::microseconds>(end - start);
```

확인 항목

- | | |
|-----------------|-------------|
| - 작업 크기 | - 원소 수 |
| - 스레드 생성과 분할 비용 | - 메모리 접근 패턴 |
| - 캐시 적중률 | - 잠금 경합 |
| - 처리 장치 사용률 | |





- 잠금을 나중에 덧붙이기보다 데이터 소유권과 작업 경계 먼저 설계
- 공유되지 않은 데이터에는 동기화 불필요
- 각 스레드가 지역 결과 계산
- 마지막에 결과 결합
- 불변 객체, 값 복사, 메시지 큐, 소유권 이동 활용

```
long long SumRange(  
    const std::vector<int>& values,  
    std::size_t first,  
    std::size_t last)  
{  
    long long total = 0;  
    for (std::size_t i = first; i < last; ++i)  
    {  
        total += values[i];  
    }  
    return total;  
}
```

```
queue.Push(std::move(task));
```

- 작업 소유권을 큐로 이동
- 생산자가 같은 작업 객체를 계속 변경하지 않도록 규칙 설정



- 스레드보다 작업을 먼저 정의
- 작업 입력 데이터
- 출력 위치
- 취소 규칙
- 완료 통지
- 소유권 이동 여부

```
struct LoadTask
{
    std::filesystem::path path;
    std::promise<Data> result;
};

void Process(std::unique_ptr<Resource> resource);

std::jthread worker { Process, std::move(resource)};
```

- **unique_ptr**: 이동으로 단일 소유권 유지
- **shared_ptr**
 - 객체 수명 공유
 - 대상 객체의 멤버 접근까지 보호하지는 않음
- 대상 객체의 가변 상태에는 별도 동기화 필요



- 각 멤버 함수가 잠금을 사용해도 여러 호출 조합은 안전 보장 안됨
- 검사와 사용 사이에 다른 스레드가 상태 변경 가능
- 복합 연산은 하나의 함수로 묶기

```
if (!queue.Empty())  
{  
    Task task = queue.Front();  
    queue.Pop();  
}
```

```
std::optional<Task> TryPop()  
{  
    std::lock_guard lock{mutex};  
    if (tasks.empty())  
    {  
        return std::nullopt;  
    }  
    Task task = std::move(tasks.front());  
    tasks.pop();  
    return task;  
}
```

스레드 A: Empty() **false** 확인
스레드 B: 마지막 원소 Pop()
스레드 A: Front() 호출

- 내부 포인터나 참조 반환은
 잠금 종료 후 위험 가능
- 값 복사 또는 소유권 이동 반환 검토



- 작업 스레드를 가진 객체는 소멸 전에 종지와 결합 필요
- 새 작업 접수 중단
- 대기자 깨우기
- 처리 중 작업 정책
- 대기 중 작업 정책
- 스레드 종료 확인

```
class Worker
{
public:
    Worker() : thread{[this](std::stop_token stopToken) {
        Run(stopToken);
    }}
    { }
```

```
private:
    void Run(std::stop_token stopToken);
    std::mutex mutex;
    std::condition_variable condition;
    std::queue<Task> tasks;
    std::jthread thread;
};
```

- 멤버 소멸은 선언 역순
- 스레드가 접근하는 데이터는 std::jthread보다 먼저 선언
- 역순 소멸에서 thread를 먼저 정리



- 스레드 수 증가 → 작업 분배와 문맥 전환 비용 증가
- 실행 가능 스레드가 처리 장치 수보다 많으면 느려질 수 있음
- `hardware_concurrency()`는 참고값
- 반환값은 0일 수 있음
- 최적 스레드 수는 작업 특성과 환경에 따라 다름

```
unsigned int count = std::thread::hardware_concurrency();  
if (count == 0)  
{  
    count = 1;  
}  
  
struct alignas(64) Counter  
{  
    std::atomic<long long> value = 0;  
};
```

- 거짓 공유는 캐시 라인 경합으로 성능 저하
- 구조 변경 전 프로파일러로 확인
- 기준은 스레드 수가 아니라 전체 처리 시간, 응답 지연, 처리량, 자원 사용량



- 동시성은 여러 작업이 겹쳐 진행되는 구조
- 병렬성은 여러 작업이 실제로 같은 시간에 실행되는 상태
- 스레드는 같은 프로세스의 메모리를 공유
- 공유 가변 상태에는 동기화 필요
- 데이터 경쟁이 있는 C++ 프로그램의 동작은 보장되지 않음
- `std::thread`는 결합 가능한 상태로 소멸하면 프로그램 종료
- `std::jthread`는 자동 결합과 중지 요청 제공
- 중지 요청은 강제 중단이 아니며 작업 함수의 협력 필요
- RAII 잠금 객체는 예외와 조기 반환에서도 잠금 해제
- 여러 뮤텝스는 잠금 순서 또는 `std::scoped_lock`으로 관리





- 조건 변수는 공유 상태의 조건이 만족될 때까지 대기
- 대기자는 조건식을 반복 확인
- future, promise, packaged_task, async → 결과와 예외를 공유 상태로 전달
- std::atomic은 한 객체에 대한 원자적 연산 제공
- 여러 값의 관계와 복합 상태 전이에는 뮤텝스가 더 명확
- 약한 메모리 순서는 정확성 증명과 성능 측정 후 적용
- latch, barrier, semaphore는 작업 진행 시점과 자원 수 조정
- 병렬 알고리즘은 독립적인 원소 작업에 적합
- 안전한 동시성 설계 → 공유 상태 축소, 소유권 분리, 종료 정책 수립

